AllMinds

Production Readiness Review

Issues found in the codebase that should be fixed before AllMinds is run as a production service, with a suggested fix for each

Neurodivergent-affirming AI teaching companion — HackAstone Agentic AI

Generated from the source of the HackAstone-Agentic-AI/AllMinds repository

1. Summary

AllMinds is a strong hackathon-grade product with a sensible architecture. This review lists **47 issues** that should be addressed to make it production grade; **12 are rated High**. The most urgent are two defects that can break a release or sign-in outright (D1 migration syntax error, D2 NOT NULL profile columns), the blocking AI endpoint (A1), the RAG index rebuild (A2), the absence of any tests or CI (F2, O1), and a handful of security design choices (S1-S5).

How this was produced. Static review of the repository source (backend, agents, frontend, migrations, deployment files). Nothing was executed against live services or with real API keys, so timings and runtime behaviour are not measured. Severity is judgement: High means data loss, account takeover, a blocked release or an outage is plausible; Medium means a real weakness with a workaround; Low means hygiene. Items citing a file are verified in that file; items in the legal/operations section are gaps observed in the repository, which may be covered outside it.

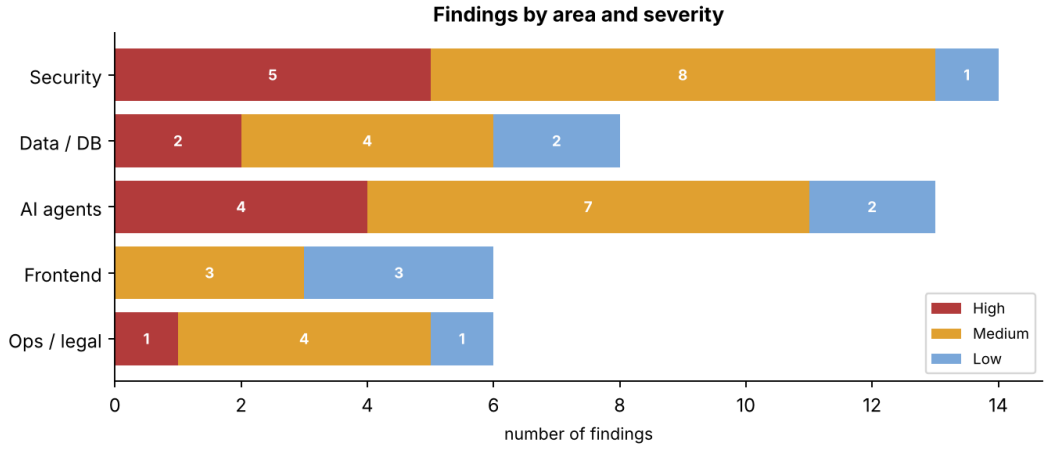


Figure 1. Distribution of findings.

What is already done well

- All SQL uses parameterised queries ($1...), including the dynamic NGO search.
- Every learner, progress, sensory and schedule query is scoped by teacher_id from the verified JWT (no insecure direct object references found).
- Reset, verification and OAuth tokens are stored only as SHA-256 hashes, single use and time-limited.
- Helmet, compression, CORS allow-list support, rate limits on auth, OAuth, reset and AI routes.
- Input and output guardrails, PII scrubbing and graceful degradation when a provider key is missing.
- Transactional registration, graceful shutdown, tracked migrations, health checks, security headers on Next.js.

Recommended order of work

When	Do this	Items
This week (quick wins)	Fix release blockers and fail-open configuration; add CI with the first tests; log AI failures; enable CSP in report-only mode.	D1, D2, S5, S8, S7, A4, O1, S3
Next 2-3 weeks	Make the AI service non-blocking with timeouts and retries; prebuild the RAG index; move tokens to cookies with refresh; encrypt calendar tokens; fix OAuth linking; turn on e-mail verification; Redis rate limits.	A1, A2, A3, S1, S2, S4, S6, S9
Next 1-2 months	Async job pattern with progress; memory with LangGraph; quotas and caching; AI evaluation set; observability stack; object storage; consolidate schema; POPIA programme.	F1, A5, A8, A10, O2, D3, D6, O6, A13
Ongoing	Dependency upgrades, accessibility, load tests, SLO reviews, incident drills.	F4, F5, O4

Table 1. Suggested roadmap

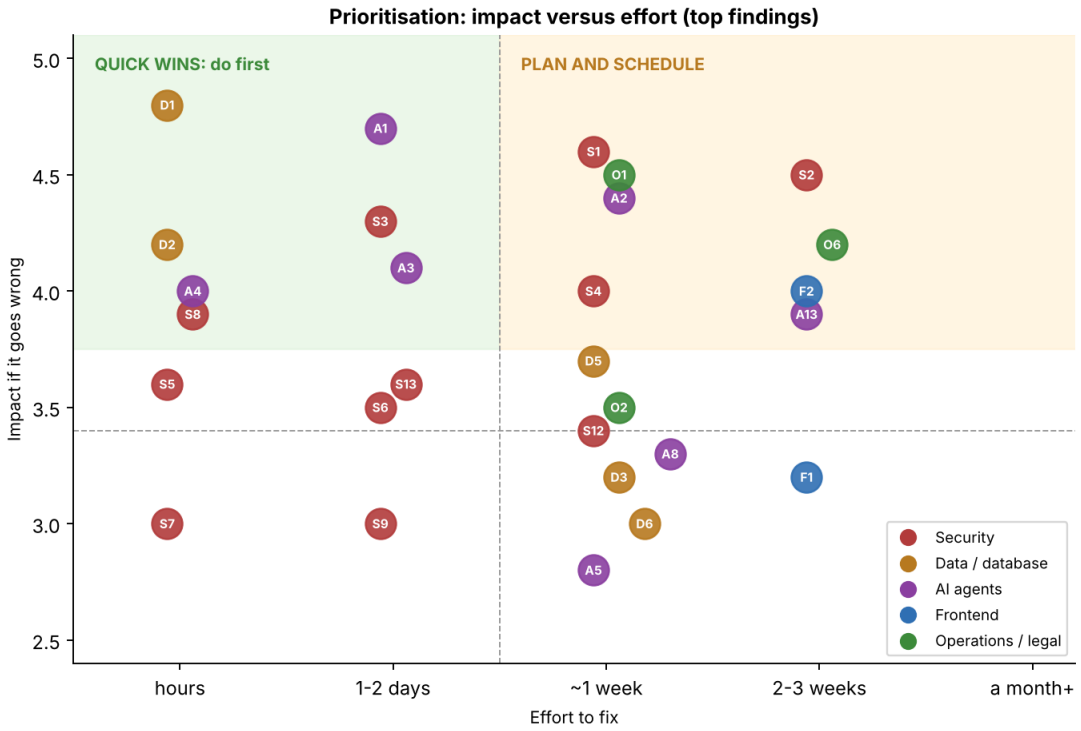


Figure 2. Quick wins are in the top-left; High-impact items needing more work sit top-right.

2. Findings and suggested fixes

2.1 Security and authentication

ID	Severity	Issue	Where	Suggested fix
S1	High	OAuth accounts are auto-linked by e-mail with no proof the provider verified it. Microsoft uses mail or userPrincipalName; every OAuth login also forces email_verified = true.	config/passport.js findOrCreateOAuthUser, Microsoft profile mapping	Require the provider's email_verified claim; add a user_identities(provider, subject) table and link on subject id, not e-mail; ask for confirmation before linking to an existing password account.
S2	High	JWT lives in localStorage for 7 days with no logout, refresh or revocation.	lib/api.ts, login/page.tsx, authController.js	httpOnly + Secure + SameSite cookie; 15-minute access token plus rotating refresh token; token_version column so a password reset or logout invalidates old tokens.
S3	High	No Content-Security-Policy anywhere (Helmet CSP disabled, none in next.config.js). Together with S2, any XSS becomes full account takeover.	app.js helmet({contentSecurityPolicy:false}); next.config.js headers	Ship a strict CSP (nonces, no inline scripts, connect-src limited to API hosts); add Trusted Types later.
S4	High	Google Calendar access and refresh tokens are stored in plaintext in Postgres.	google_calendar_tokens; calendarController.js; calendar_token_store.py	Encrypt with AES-256-GCM using a key from a secrets manager (or pgcrypto + KMS); rotate keys; request the narrowest scope (calendar.events instead of full calendar).
S5	High	/api/stats/admin has no authorizeAdmin and reads users.school_id, a column that does not exist, so it fails for everyone.	routes/statsRoutes.js; statsController.js getSchoolIdByAdmin	Add authorizeAdmin, add users.school_id (FK to schools) via a migration, and add a test for 403 vs 200.
S6	Medium	E-mail verification is switched off, so anyone can register with any address (squatting, spam, fake accounts).	authController.register; emailVerificationController.js	Configure a transactional e-mail provider (SES, Postmark, Resend), turn the gate back on, add SPF/DKIM/DMARC.
S7	Medium	Database TLS accepts any certificate (rejectUnauthorized:false).	config/database.js	Use the provider's CA bundle with verification on, or Render's private network URL.
S8	Medium	Configuration fails open: CORS allows every origin if ALLOWED_ORIGINS/FRONTEND_URL are empty; missing URLs silently default to localhost; only JWT_SECRET is checked and not for strength.	app.js CORS callback; config/env.js	In production refuse to start unless required variables are present; require JWT_SECRET >= 32 random bytes; validate with zod/envalid.
S9	Medium	Rate limiters use the default in-memory store: counters are per instance and reset on every restart, so limits weaken as soon as you run two instances.	express-rate-limit usage in routes/*.js	Use a Redis store; add per-account lockout/backoff on repeated failed logins.
S10	Medium	bcrypt only reads the first 72 bytes but passwords up to 128 characters are accepted; there is no breached-password check.	authController.js, passwordResetController.js	Cap at 72 bytes or pre-hash with SHA-256, or move to argon2id; check the Have I Been Pwned k-anonymity API.
S11	Medium	The same JWT_SECRET signs login tokens and the calendar OAuth state HMAC (key reuse).	calendarController.js buildState	Use separate secrets per purpose (derive with HKDF).
S12	Medium	Prompt-injection guard only inspects the chat message. learner_name, records[], support_need and other structured fields go into prompts unchecked.	api/index.py ChatRequest; agents	Validate with strict pydantic models, length-limit and strip control text, wrap user data in delimiters and tell the model to treat it as data; add an injection test suite.
S13	Medium	A teacher who writes 'my student talks about self-harm' is blocked by the regex with a generic refusal, exactly when a safeguarding answer is needed.	core/guardrails.py _HARM_PATTERNS	Split 'unsafe request' from 'safeguarding concern': respond with a vetted script (who to tell, SA helplines, school child-protection procedure) instead of a refusal; log for review.

ID	Severity	Issue	Where	Suggested fix
S14	Low	Reset links are returned in the API response whenever NODE_ENV is not 'production' and SMTP is missing; a mis-set staging environment would leak them.	passwordResetController.js	Gate on an explicit ALLOW_DEV_RESET_LINK flag that is never set outside local development.

Table 2. Security findings

2.2 Database and data integrity

ID	Severity	Issue	Where	Suggested fix
D1	High	Migration 20260629 uses ADD CONSTRAINT IF NOT EXISTS, which PostgreSQL does not support. Because Render runs db:migrate before every deploy, this blocks releases on a clean database.	migrations/20260629_forum_comments_and_schedule_unique.sql	Wrap in a DO $$ ... IF NOT EXISTS (SELECT 1 FROM pg_constraint WHERE conname = ...) $$ block; run all migrations on an empty database in CI.
D2	High	teacher_profiles.name and surname are NOT NULL, but register and OAuth insert NULL when a value is missing (a single-word Google display name gives an empty surname), so sign-in can fail with a constraint error.	docs/database schema lines 185-186; authController.js; passport.js parseName	Make the columns nullable or default to '', and add a test for single-name and missing-name sign-ups.
D3	Medium	Three sources of truth for the schema (docs/database, README/ copy with a different hash, migrations). The baseline lacks email_verified; init-db skips if users already exists.	README/allminds_complete_schema.sql; docs/database; scripts/init-db.js	One baseline plus numbered, immutable migrations (node-pg-migrate or Flyway); delete the duplicate; CI check that a fresh database equals the migrated one.
D4	Medium	E-mail case is inconsistent: register lowercases, OAuth inserts and looks up the raw address, login uses email = $1. Duplicate or unreachable accounts are possible.	passport.js; authController.js	Use citext or a unique index on lower(email); normalise at every entry point.
D5	Medium	Teacher data has no deletion or export path (POPIA right to erasure and access); no backup, point-in-time-recovery or restore drill is documented.	no endpoint; render.yaml basic-256mb plan	DELETE /api/profile (cascades already exist), data-export endpoint, retention policy; enable PITR and test a restore.
D6	Medium	Uploaded curriculum documents are written to local disk (multer diskStorage), which Render wipes on every deploy; no malware scan.	schoolController.js	Store in S3-compatible object storage with signed URLs and a virus scan; keep only metadata in Postgres.
D7	Low	Pool of 20 connections per instance can exceed the connection cap of a small Postgres plan; 2 s connect timeout is tight; no statement_timeout.	config/database.js	Put PgBouncer in front, size the pool from the plan, set statement_timeout and idle_in_transaction timeouts.
D8	Low	language_pref defaults to 'English' in the schema but the code uses codes such as 'en', 'zu'.	users.language_pref	Default to 'en' and add a CHECK against the 11 supported codes.

Table 3. Database findings

2.3 AI agents (reliability, cost, quality, privacy)

ID	Severity	Issue	Where	Suggested fix
A1	High	Blocking LLM calls inside an async FastAPI handler. While one 60-second lesson plan runs, the event loop is blocked, so /health and every other request stall on that worker.	api/index.py chat_endpoint -> run_multi_agent (sync)	Declare the endpoint with def (thread pool) or use run_in_threadpool / async LLM clients; run several Uvicorn workers; make /health independent of work.

ID	Severity	Issue	Where	Suggested fix
A2	High	RAG index is rebuilt with Chroma.from_documents on every process start. On a persisted directory this re-adds the same chunks (duplicate hits, growing store) and causes a 30-120 s cold start; the folder is also on ephemeral disk.	agents/rag_agent.py _get_db	Build the index once at image build time (or a CI job), load with Chroma(persist_directory=...), use deterministic chunk ids, warm it at startup and report readiness in /health.
A3	High	No per-call timeouts, retries, circuit breaker or fallback model. Only the outer 180 s limit protects the request; one slow provider ties up a worker for minutes.	all agents (llm.invoke)	Set request_timeout and max_retries on each client, add tenacity back-off, circuit-break a failing provider, and fall back Pro -> Flash for lesson plans.
A4	High	Failures are invisible: run_multi_agent catches every exception, logs nothing, and guesses the cause by looking for 'api', 'key' or 'connect' in the message.	core/router.py except block	logger.exception with request id, user id (hashed) and agent; return stable error codes; alert on rate.
A5	Medium	Conversation memory is not wired in; the SQLite file uses a relative path, opens a connection per call and would be lost on redeploy.	core/memory_tools.py	Add langgraph + langgraph-checkpoint-sqlite (or use Postgres), absolute MEMORY_DB_PATH on a persistent disk, scrub PII before saving, expose a delete-my-memory call.
A6	Medium	Analytics charts are written to a relative 'charts/' folder named after the learner label: two teachers with the same label overwrite each other's files, files are never deleted and the paths are never returned to the UI.	core/analytics_automation.py build_charts	Render to in-memory buffers (or a temp dir with uuid names, cleaned up) and return them, or remove the dead code because the frontend draws its own charts.
A7	Medium	Images are generated one after another and returned as base64 data URIs inside the JSON (megabytes per response, slow, uncacheable).	core/visual_media.py	Generate in parallel, upload to object storage and return URLs; cache by topic+grade.
A8	Medium	Cost control is limited to 60 requests / 15 min per user. No daily token or spend budget, no caching of repeat curriculum questions, visual agent may call the LLM twice.	agentRoutes.js; visual_support_agent.py retry	Per-user and per-school daily quotas, response cache for RAG and common prompts, spend alerts at the provider.
A9	Medium	Every chat does an extra HTTP call to Express to validate the JWT, making Express a single point of failure for the AI path.	api/index.py get_authenticated_user_id	Verify the JWT locally (RS256 public key) or cache validation for 60 s.
A10	Medium	No automated quality evaluation: routing accuracy, curriculum correctness and neurodivergent-affirming tone are untested; RAG answers return no source page citations.	no eval set; rag_agent.py	Build a golden set of 100+ prompts, run it in CI with LLM-as-judge plus human review, return page numbers from retrieved chunks.
A11	Low	apps/agents/vercel.json routes everything to a Vercel Python function; serverless time and size limits cannot host 25-90 s generations plus torch and ChromaDB.	apps/agents/vercel.json	Delete it or document that Render is the only supported host.
A12	Low	Dependencies use ranges with no lock file (pip) so builds are not reproducible; sqlalchemy is listed but only an unused model imports it.	requirements.txt	Lock with uv or pip-tools, use CPU-only torch, drop unused packages, add pip-audit.
A13	Medium	Learner and teacher text is sent to Google and Groq (outside South Africa). The repo has no data-processing agreement, consent text or opt-out.	all agents	Use paid/enterprise API tiers whose terms exclude training on your data, sign DPAs, document cross-border transfer under POPIA section 72, add consent and a 'do not enter identifying details' notice.

Table 4. Agent findings

2.4 Frontend and user experience

ID	Severity	Issue	Where	Suggested fix
F1	Medium	Long AI requests are a single blocking fetch (up to 185 s) with no cancel, progress or resume; intermediate proxies and mobile networks commonly drop idle connections well before that.	lib/api.ts chatWithAgent; route.ts	Job pattern: POST returns 202 + id, UI polls or listens over SSE; add an abort button and stream partial text.
F2	Medium	No automated tests of any kind in the repository (no unit, API, component or end-to-end tests), so every change is verified by hand.	whole repo	Jest + Supertest (backend), pytest with mocked LLMs (agents), Vitest/RTL and Playwright (frontend).
F3	Medium	Very large page components (learnerAnalytics 1021 lines, assessment-planner 789) and widespread 'any' types; no error boundaries or error.tsx.	app/dashboard/**	Split into hooks and components, generate types from an OpenAPI spec, add error.tsx/not-found.tsx.
F4	Low	Next.js 13.5 is two majors behind; no dependency or licence scanning.	package.json	Upgrade to a supported Next.js, enable Dependabot and npm audit in CI.
F5	Low	Accessibility is not verified even though the audience is neurodivergent teachers; jsPDF default fonts may not render every SA-language character (e.g. Tshivenda).	components, exports	axe-core in CI, WCAG 2.2 AA review, embed a Unicode font (Noto Sans) in jsPDF.
F6	Low	Low-bandwidth and offline behaviour: large data-URI images, no caching or PWA support for schools with poor connectivity.	image_aids handling	Serve images by URL with CDN caching; PWA shell.

Table 5. Frontend findings

2.5 Operations, delivery and governance

ID	Severity	Issue	Where	Suggested fix
O1	High	No CI/CD: no pipeline for lint, test, build, migration test, dependency audit or secret scanning; no staging environment.	no .github/ directory	GitHub Actions: lint + test + build on PR, gitleaks, npm audit / pip-audit, deploy to staging, smoke test, manual promote to production.
O2	Medium	Observability gap: 83 console.log/error calls, no structured logs, request ids, metrics, tracing or alerting. /api/health always says OK without touching the database; the agent /health does not check models or the vector index.	app.js; api/index.py	pino/structlog JSON logs with request id propagated across the three services, Sentry, OpenTelemetry, uptime checks, readiness endpoints that test DB and providers.
O3	Medium	No .env.example and no Dockerfiles, so onboarding and environment parity depend on tribal knowledge; agent image will be large (torch + sentence-transformers).	repo root	Commit .env.example per service, multi-stage Dockerfiles, docker-compose for local dev, pre-download the embedding model in the image.
O4	Medium	Single region, single instance per service and no autoscaling policy; no documented SLOs, runbook or incident process.	render.yaml	Define SLOs (e.g. p95 chat latency, 99.5% availability), 2+ instances for stateless services, runbooks, on-call alert routes.
O5	Low	Repository hygiene: legacy demo HTML inside backend/src, README/ folder holding a schema copy, stale system-design.json (says Next.js calls FastAPI directly), file names with typos (lesson_planer_agent.py).	apps/backend/src/frontend-demo; system-design.json	Delete or move demo files, update the design doc, rename files.

ID	Severity	Issue	Where	Suggested fix
O6	Medium	Legal and governance: no privacy policy, terms, consent flow, records of processing, information-officer details or AI-use disclaimer ('not a diagnostic tool; not a substitute for an educational psychologist').	frontend, docs	POPIA gap assessment, privacy notice and consent on sign-up, DPA with schools, appoint an information officer, show disclaimers on every AI result.

Table 6. Operations findings

3. What 'production grade' should look like

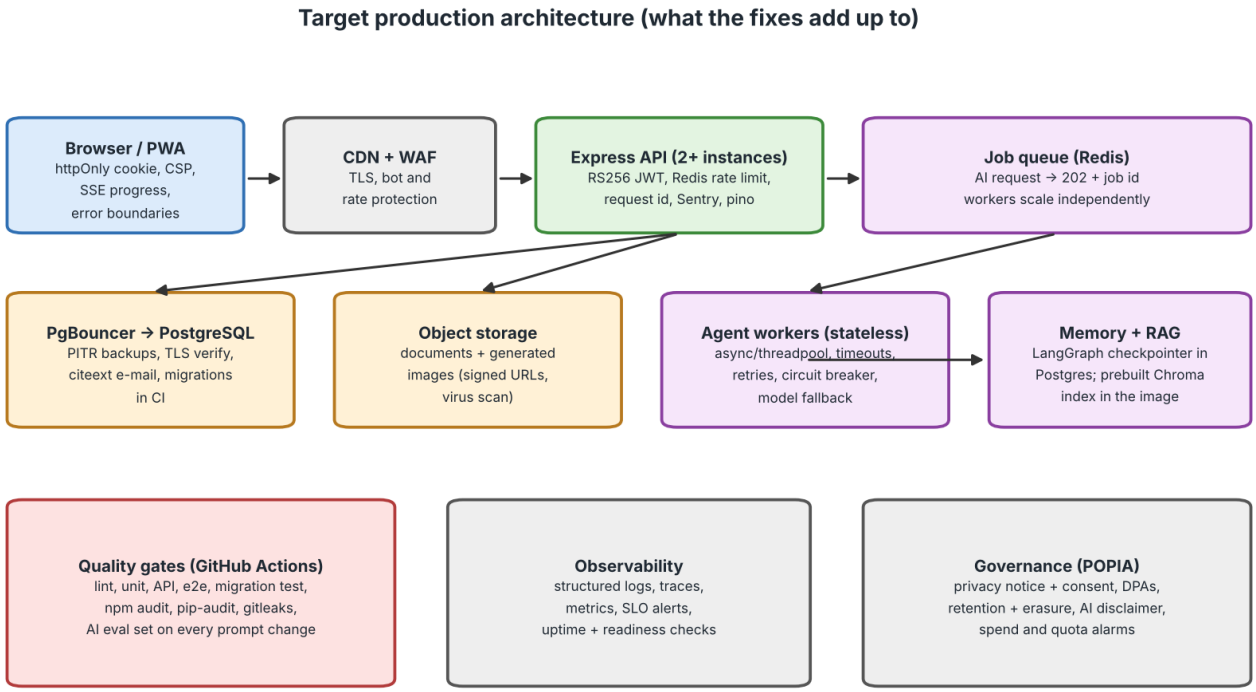


Figure 3. The target state the fixes converge on.

3.1 Launch checklist

Area	Done when
Release	Fresh database migrates cleanly in CI; staging mirrors production; one-click rollback.
Security	Cookie auth with refresh, CSP enforced, tokens encrypted, OAuth linked by subject id, secrets in a manager, dependency and secret scans green.
Reliability	AI calls have timeouts/retries/fallbacks; /health reflects real readiness; load test shows target p95 at expected concurrent teachers.
Data	Backups with a tested restore, PITR on, erasure and export endpoints, retention schedule.
AI quality	Evaluation set passes thresholds; safeguarding path for self-harm/abuse; clear disclaimers; cost alarms.
Compliance	POPIA review complete, privacy notice and consent live, DPAs signed with Google, Groq and the hosting provider.
Operations	Dashboards, alerts, runbooks, on-call owner, incident template.

Table 7. Launch checklist

4. Hackathon final - questions about production readiness

Q1. Is this ready for real schools tomorrow?

It is a working, well-structured product with good fundamentals (parameterised SQL, per-teacher data scoping, hashed tokens, guardrails). For paying schools we would first close the High items: release blockers, non-blocking AI service, tests and CI, and the token/CSP hardening. That is roughly a four-to-six-week plan, not a rewrite.

Q2. What is your biggest technical risk?

Reliability of the AI path under concurrency: synchronous model calls on an async server plus a cold RAG index. We would fix it with a threadpool or job queue, prebuilt index, timeouts and a fallback model.

Q3. How do you protect children's data?

Learners are pseudonymised, every query is scoped to the owning teacher, PII is scrubbed before any model call and tokens are hashed. Production needs encryption of calendar tokens, a retention and erasure process, signed processing agreements with AI providers, and POPIA consent.

Q4. How would you know the AI is good enough?

A golden evaluation set run in CI on every prompt or model change: routing accuracy, curriculum correctness against DCAPS with citations, affirming-language checks, safety refusals, plus periodic review by an educational psychologist.

Q5. What happens when Gemini or Groq is down?

Today: friendly error and keyword routing without Groq. Target: retries, circuit breaker, Pro-to-Flash fallback, queued jobs and a status banner, so teachers are never left with a spinner.

Q6. How will it scale to thousands of teachers?

Stateless services behind a load balancer, PgBouncer for Postgres, Redis for rate limits and queues, object storage for files and images, AI workers scaled separately, and per-user quotas plus caching to control spend.

Q7. What would you do in the first week with the team?

Fix D1/D2, put CI in place with the first API tests, add structured error logging to the agents, enforce fail-fast configuration, and ship CSP in report-only mode.

Q8. Why not just rewrite it?

The architecture (Next.js, Express/Postgres, FastAPI agents) is sound and the problems are localised. Targeted fixes are cheaper and keep the product moving.